Quickstarts

Transfer USDC or EURC

Copy page

Learn how to transfer USDC or EURC from one wallet address to another

This tutorial walks you through transferring USDC or EURC from one wallet address to another on Arc Testnet. Select the tab that matches your preferred setup.

Circle Wallets Viem

Prerequisites

Before you begin, make sure you have:

1. A **Circle Developer Console** account
2. An API key created in the Console:
   Keys → Create a key → API key → Standard Key
3. Your **Entity Secret registered**

Step 1. Set up your project

In this step, you set up your local development environment and install the required dependencies.

1.1. Create the project and install dependencies

Create a new directory, navigate to it and initialize a new project.

Node.js Python

```
mkdir transfer-funds && cd transfer-funds
npm init -y
npm pkg set type=module
```

In the project directory, install the Circle Dev-Controlled Wallets SDK. It is also possible to call the API directly if you can't use the SDK in your project.

Node.js Python

```
npm install @circle-fin/developer-controlled-wallets
```

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

Shell

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Set environment variables

Create a `.env` file in the project directory with your Circle credentials, replacing these placeholders with your own credentials:

.env

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
```

- `CIRCLE_API_KEY` is your Circle Developer API key for Wallets API requests.
- `CIRCLE_ENTITY_SECRET` is your registered entity secret used to authorize developer-controlled wallet operations.

In the Node.js examples, npm commands can load variables from `.env` using Node.js native env-file support.

> Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Set up your wallets

In this step, you create two Arc Testnet dev-controlled wallets and fund one of them with testnet USDC or EURC for the transfer flow. If you already have two Arc Testnet dev-controlled wallets and one is funded with USDC or EURC, skip to **Step 3**.

2.1. Create wallets

Import the Wallets SDK and initialize the client using your API key and Entity Secret. Dev-controlled wallets are created in a **wallet set**, which serves as the source from which individual wallet keys are derived.

Node.js Python

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer
                                                        ›
const client = initiateDeveloperControlledWalletsClient({
  apiKey: "<YOUR_API_KEY>",
  entitySecret: "<YOUR_ENTITY_SECRET>",
});

// Create a wallet set
const walletSetResponse = await client.createWalletSet({
  name: "Wallet Set 1",
});

// Create a wallet on Arc Testnet
const walletsResponse = await client.createWallets({
  blockchains: ["ARC-TESTNET"],
  count: 2,
  walletSetId: walletSetResponse.data?.walletSet?.id ?? "",
});
```

> If you're calling the API directly, you'll need to make two requests: one to **create the wallet set**; one to **create the wallet**.
>
> Be sure to replace the **Entity Secret ciphertext** and the idempotency key in your request. If you're using the SDKs, this is handled automatically for you.

You should have two new Externally Owned Account (EOA) developer-controlled wallets that you can also see from the Circle Developer Console. The API response will look similar to the following:

```
  {
    "id": "a2f67c91-b7e3-5df4-9c8e-42bbd51a9fcb",
    "state": "LIVE",
    "walletSetId": "5c3e9f20-8d4b-55a1-a63b-c21f44de8a72",
    "custodyType": "DEVELOPER",
    "refId": "",
    "name": "",
    "address": "0x9eab451f27dca39bd3f5d76ef28c86cc0b3a72df",
    "blockchain": "ARC-TESTNET",
    "accountType": "EOA",
    "updateDate": "2025-11-07T01:35:03Z",
    "createDate": "2025-11-07T01:35:03Z"
  },
  {
    "id": "c84d12a4-f6a9-5df8-8b44-92ff7cc94e32",
    "state": "LIVE",
    "walletSetId": "5c3e9f20-8d4b-55a1-a63b-c21f44de8a72",
    "custodyType": "DEVELOPER",
    "refId": "",
    "name": "",
    "address": "0xb37ac90d1a657c04a8518b6f7bda2b37e4f8d221",
    "blockchain": "ARC-TESTNET",
    "accountType": "EOA",
    "updateDate": "2025-11-07T01:35:06Z",
    "createDate": "2025-11-07T01:35:06Z"
  }
]
```


2.2. Fund a wallet with testnet stablecoins

Obtain testnet USDC or EURC from the Circle Faucet or the Console Faucet to perform actions like making transfers and paying gas fees. You'll need a funded balance to execute transactions using your dev-controlled wallet.

2.3. Check the wallet balances

You can check your wallet balances from the Circle Developer Console or programmatically by making a request to GET /wallets/{id}/balances with the specified wallet ID.

Node.js Python cURL

```
const response = await client.getWalletTokenBalance({
  id: "<WALLET_ID>",
});
```

Step 3. Transfer USDC / EURC

In this step, you set up your script, execute the USDC or EURC transfer, and check the result.

3.1. Set up the transfer script and execute the transfer

Replace the `walletAddress` field with the address of the funded wallet, and the `destinationAddress` field with the address of the other wallet.

USDC EURC

Node.js Python cURL

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer
                                                        ›
const client = initiateDeveloperControlledWalletsClient({
  apiKey: "<YOUR_API_KEY>",
  entitySecret: "<YOUR_ENTITY_SECRET>",
});

const transferResponse = await client.createTransaction({
  amount: ["0.1"], // Transfer 0.1 USDC
  destinationAddress: "<RECIPIENT_ADDRESS>",
  tokenAddress: "0x3600000000000000000000000000000000000000", // USDC contract
  blockchain: "ARC-TESTNET",
  walletAddress: "<SENDER_ADDRESS>",
  fee: {
    type: "level",
    config: {
      feeLevel: "MEDIUM",
    },
  },
});
console.log(transferResponse.data);
```

If the script runs successfully, you should receive a response object with a transaction ID and the state of the transaction that looks similar to the following:

```
{ "id": "70cb796b-5c43-5076-a790-3525f6d8424c", "state": "INITIATED" }
```

3.2. Verify the status of the transfer

You can check the status of the transfer with the `getTransaction` method by providing the transaction ID obtained from the previous step.

Node.js Python cURL

```
const response = await client.getTransaction({
  id: "<TRANSACTION_ID>",
});
console.log(response.data);
```

You can verify that the transaction was successful by checking the details in the returned object, which should look similar to the following:

```
{
  "data": {
    "transaction": {
      "id": "70cb796b-5c43-5076-a790-3525f6d8424c",
      "blockchain": "ARC-TESTNET",
      "tokenId": "15dc2b5d-0994-58b0-bf8c-3a0501148ee8",
      "walletId": "f8a9993d-5a59-58df-b3e0-206c6e81ea57",
      "sourceAddress": "0x3bafa3a0987699391a2003c3aa06d41b0209c397",
      "destinationAddress": "0x1f5ee284571f4bb888ceda4cc7b6e5e15829a1ea",
      "transactionType": "OUTBOUND",
      "custodyType": "DEVELOPER",
      "state": "COMPLETE",
      "transactionScreeningEvaluation": {
        "screeningDate": "2026-01-07T04:49:09Z"
      },
      "amounts": [
        "0.1"
      ],
      "nfts": null,
      "txHash": "0x2e312718a2b6663cf20862e71b52dcb5404c9dd16510a08c5c9f47a78aab
      "blockHash": "0x7474b111d671de75e7b606b8e1a1508c9bcef0eafc718843accb90051
      "blockHeight": 20375164,
      "networkFee": "0.003556682015901",
      "firstConfirmDate": "2026-01-07T04:49:11Z",
      "operation": "TRANSFER",
      "feeLevel": "MEDIUM",
      "estimatedFee": {
        "gasLimit": "21000",
        "networkFee": "0.006916682015901",
        "baseFee": "160",
        "priorityFee": "9.365810281",
        "maxFee": "329.365810281"
      },
      "refId": "",
      "abiParameters": null,
      "createDate": "2026-01-07T04:49:09Z",
      "updateDate": "2026-01-07T04:49:11Z"
    }
  }
}
```

You can also copy the transaction hash (`txHash`) value and look up the transaction details on the **Arc Testnet explorer**.

Summary

After completing this tutorial, you've successfully:

- Set up a wallet workflow for either Circle Wallets or a self-managed wallet
- Prepared your wallet with testnet tokens
- Transferred USDC or EURC from one wallet to another

Was this page helpful? Yes No

Bridge USDC to Arc

Access USDC crosschain

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.